

ISIS 2111 – Elementos Esenciales de Lenguajes de Programación

Proyecto 4

Estudiantes:

- Ana María Sandoval Jimenez - 202420984
- Juan Esteban García Bonilla - 202420481

1. Introducción

En este proyecto se continuó con la implementación del lenguaje VeriLang en Rascal, tomando como base lo hecho en el Proyecto 3. En esa entrega el sistema ya podía leer archivos .vl, construir el AST, validar tipos y reglas semánticas, y generar una salida en texto del programa.

El objetivo del Proyecto 4 fue reutilizar esa implementación cambiando el lenguaje destino a Kotlin. Es decir, VeriLang sigue siendo definido y procesado desde Rascal, pero el resultado ahora se puede ejecutar desde el entorno de Kotlin.

Para esto se mantuvieron los módulos principales del Proyecto 3 (Syntax.rsc, AST.rsc, Parser.rsc, Implode.rsc, TypeChecker.rsc y Main.rsc) y se agregó un nuevo módulo llamado KotlinGenerator.rsc, encargado de transformar la representación de VeriLang en un archivo Kotlin ejecutable.

Como resultado, el proyecto permite tomar un programa VeriLang válido, verificarlo, generar un archivo FullProgram.kt, compilarlo con kotlinc y ejecutarlo en Kotlin/JVM.

2. Cambios respecto al Proyecto 3

Se partió de la solución funcional del Proyecto 3, que ya tenía parser, AST, generación textual, integración con TypePal y un TypeChecker para validar tipos, operadores, espacios, variables e identificadores.

El primer cambio fue adaptar las rutas internas del proyecto, actualizando todas las referencias de `project://ple_project2` a `project://ple_project4`. Esto se aplicó en Main.rsc, Generator.rsc, VeriLangTypePal.rsc, Plugin.rsc y KotlinGenerator.rsc.

El segundo cambio fue corregir Implode.rsc. Durante las pruebas apareció un problema de ambigüedad con expresiones como `(implication p q)`: Rascal a veces la interpretaba como expresión infija en vez de aplicación prefija, generando errores como `Undefined infix operator: p`. Se resolvió haciendo que cuando hay ambigüedad, el sistema priorice la alternativa `application`.

El tercer cambio fue agregar el módulo KotlinGenerator.rsc, que carga `fullProgram.vl`, corre el TypeChecker y, si no hay errores, genera `FullProgram.kt`.

También se agregó la carpeta `instance/kotlin/` para guardar los archivos generados: `FullProgram.kt` y `FullProgram.jar`.

3. Reutilización de VeriLang

El Proyecto 4 reutiliza directamente la implementación del Proyecto 3. No se rehizo nada desde cero. Los módulos que se conservaron son:

- Syntax.rsc
- AST.rsc
- Parser.rsc
- Implode.rsc
- Generator.rsc
- TypeChecker.rsc
- VeriLangTypePal.rsc
- Main.rsc

Gracias a esto se mantuvo la misma sintaxis de VeriLang, el mismo formato de archivos .vl y las mismas reglas de validación.

El archivo de prueba principal siguió siendo fullProgram.vl, que tiene un programa válido con espacios, operadores, variables, una regla lógica y cuantificadores. También se mantuvo invalidProgram.vl para probar que el sistema rechaza programas con errores.

4. Corrección de ambigüedad en Implode

Al probar el proyecto se encontró que la expresión (implication p q) a veces era interpretada de dos formas distintas. La correcta es como aplicación prefija: application(implication, [p, q]). Pero Rascal a veces la trataba como expresión infija, donde p aparecía como operador, lo que generaba errores como:

```
Undefined infix operator: p
Undefined identifier: implication
```

Para corregirlo se modificó Implode.rsc con las funciones isApplicationTree y chooseAmbAlternative. Cuando hay ambigüedad, el sistema revisa si alguna alternativa es una application y la escoge primero. Con este cambio las expresiones prefijas de VeriLang se procesan de forma estable.

5. Implementación de KotlinGenerator.rsc

El cambio principal del Proyecto 4 fue crear el módulo KotlinGenerator.rsc. Su flujo es:

1. Lee el archivo fullProgram.vl.
2. Construye el AST con loadModule.
3. Corre el TypeChecker con checkModule.
4. Si no hay errores, genera FullProgram.kt.
5. Si hay errores, no genera nada.

La función principal es main() y también existe testInvalid(), que intenta generar Kotlin desde invalidProgram.vl para demostrar que el sistema rechaza programas con errores antes de producir código.

Para generar el código Kotlin, el módulo reutiliza Generator.rsc: obtiene la representación textual del módulo, la divide por líneas y envuelve cada una en un println(...). Además, la función escapeKotlinString se encarga de escapar correctamente los caracteres especiales de Kotlin (\, ", \$) para que el archivo .kt generado sea válido.

6. Ejecución en Kotlin

Después de generar FullProgram.kt en instance/kotlin/, se compiló con:

```
kotlinc          instance/kotlin/FullProgram.kt          -include-runtime          -d
instance/kotlin/FullProgram.jar
```

Y se ejecutó con:

```
java -jar instance/kotlin/FullProgram.jar
```

La ejecución imprimió correctamente la representación del programa VeriLang en consola, con el módulo, imports, espacios, operadores, variables, reglas y expresiones. Esto confirmó que el código generado desde Rascal compila y corre bien en el runtime de Kotlin/JVM.

7. Validación con programa válido e inválido

Se usaron dos archivos para validar el sistema: fullProgram.vl e invalidProgram.vl.

Al correr KotlinGenerator::main() sobre el programa válido, la salida fue:

```
Parsed successfully: Set
Type check successful.
No semantic errors found.
Kotlin file generated at: |project://ple_project4/instance/kotlin/FullProgram.kt|
```

El programa inválido tiene estos errores intencionales:

```
defvar p : Bool, q : Bool, r : Perro end
defrule (implication p z) -> (disjunction (negation p) q) end
defexpression (forall x in Element . (forall S in Set . (isIn x W))) end
```

Los errores esperados son que Perro no existe como tipo, z no fue declarado y W no fue declarado. Al correr KotlinGenerator::testInvalid(), la salida fue:

```
Parsed successfully: Set
Type check failed.
Semantic errors found:
- Undefined type: Perro
- Undefined identifier: z
- Undefined identifier: W
Kotlin file was not generated because the program has semantic errors.
```

Esto confirma que el generador valida el programa antes de producir código: acepta los válidos y rechaza los que tienen errores.

8. Pruebas finales realizadas

Las pruebas se hicieron en dos partes: desde el REPL de Rascal y desde la terminal.

Primero se verificó que el proyecto base siguiera funcionando:

```
import Main;
Main::main();

Parsed successfully: Set
Type check successful.
No semantic errors found.
Output generated at: |project://ple_project4/instance/output/fullProgram.txt|
```

Luego se probó la generación de Kotlin:

```
import KotlinGenerator;
KotlinGenerator::main();
```

El sistema generó correctamente FullProgram.kt. Después se probó el caso inválido:

```
KotlinGenerator::testInvalid();
```

El sistema detectó los tres errores semánticos y no generó código. Finalmente se compiló y ejecutó el JAR, confirmando la salida correcta en consola. El flujo completo verificado fue:

VeriLang (.vl) → Rascal → AST → TypeChecker → Kotlin (.kt) → JAR → ejecución

9. Archivos modificados/agregados

Los archivos modificados o agregados fueron:

- Implode.rsc: se ajustó para resolver ambigüedades priorizando aplicaciones prefijas.
- KotlinGenerator.rsc: módulo nuevo que genera código Kotlin a partir de un programa VeriLang válido.
- Main.rsc: actualizado con las rutas del Proyecto 4.
- Generator.rsc: reutilizado para obtener la representación textual de VeriLang.
- TypeChecker.rsc: reutilizado para validar el programa antes de generar Kotlin.
- VeriLangTypePal.rsc: mantenido como integración del sistema de tipos.
- fullProgram.vl: programa válido usado como entrada principal.
- invalidProgram.vl: programa inválido para probar el rechazo de errores.
- FullProgram.kt: archivo Kotlin generado.
- FullProgram.jar: ejecutable producido desde el código Kotlin.

10. Nota sobre el helper de Kotlin

El enunciado menciona un archivo auxiliar VeriLangService.kt con una GUI que debería mostrar el estado del parser, la lista de módulos y otra información relevante. Sin embargo, ese archivo no estaba disponible en los recursos entregados para esta práctica.

Por eso se implementó KotlinGenerator.rsc directamente en Rascal. Este módulo cumple los mismos requisitos funcionales: muestra **ok** para archivos correctos y **fail** con los errores para archivos incorrectos, y lista los módulos del programa procesado, como se ve en las secciones 7 y 8.

11. Conclusión

En este proyecto se extendió VeriLang para generar y ejecutar código Kotlin. Se reutilizaron el parser, el AST, el sistema de tipos y la validación semántica del Proyecto 3, y se agregó el nuevo módulo KotlinGenerator.rsc.

El resultado permite leer un archivo .vl, validarlo, generar un .kt, compilarlo con kotlinc y ejecutarlo en Kotlin/JVM. El sistema rechaza programas con errores antes de generar código.

Las pruebas muestran que fullProgram.vl genera correctamente FullProgram.kt y que el JAR corre sin errores, mientras que invalidProgram.vl es rechazado con los tres errores esperados. Con esto el proyecto cumple el objetivo de reutilizar VeriLang desde Rascal y desplegarlo hacia Kotlin.